

TREBALL DE FINAL DE GRAU

Hefest: disseny i avaluació d'un motor de jocs en C amb Vulkan

[Títol acordat amb el director.]

Oleguer Almuni
Grau en Enginyeria Multimèdia

11/09/2026

Resum

[Màxim una pàgina. Ha de contenir, en aquest ordre (ho exigeix la plantilla):

- 1. Plantejament del problema o de la necessitat a cobrir.*
- 2. Objectiu del treball.*
- 3. Resultats obtinguts que responen a aquest problema.*
- 4. Conclusions obtingudes.*

Escriure'l l'últim, quan els resultats ja existeixin. És el primer que llegeix el tribunal i sovint l'únic que llegeixen sencer abans de la defensa.]

Agraïments

[Opcional segons la plantilla.]

Índex

Acrònims	8
1 Introducció	1
1.1 Context i motivació	1
1.2 Objectius	1
1.3 Abast	1
1.4 Organització del document	2
2 Estat de l'art i marc tecnològic	3
2.1 Arquitectura d'un motor de jocs	3
2.1.1 Les capes d'un motor 3D típic	3
2.1.2 Correspondència amb el motor desenvolupat	4
2.2 De OpenGL a Vulkan: per què existeix Vulkan	5
2.2.1 El model clàssic i el seu cost	5
2.2.2 Un primer intent des de dins: AZDO	5
2.2.3 Mantle: traslladar la responsabilitat a l'aplicació	6
2.2.4 De Mantle a Vulkan	6
2.2.5 El principi de control explícit	7
2.2.6 Conseqüència: l'API imposa l'arquitectura	8
2.3 Motors existents i per què construir-ne un	8
2.3.1 El llinatge com a norma en el desenvolupament de motors	9
2.4 Projectes i materials de referència	9
3 Metodologia i planificació	10
3.1 Metodologia de desenvolupament	10
3.1.1 Fases del desenvolupament	10
3.1.2 Verificació	10
3.1.3 Documentació com a part del mètode	11
3.2 Procedència de les fonts i ús de projectes de referència	11
3.2.1 Base emprada	11
3.2.2 Aportacions pròpies	11
3.2.3 Delimitació	12
3.3 Planificació temporal	12
3.3.1 Premissa de disponibilitat	12
3.3.2 Pressupost d'hores	12
3.3.3 Fases, càrrega i temporització	13
3.3.4 Desviacions	13
3.4 Eines i entorn de desenvolupament	13
3.4.1 Llenguatge, compilació i construcció	13
3.4.2 API gràfica i eines associades	13
3.4.3 Control de versions i documentació	14
3.4.4 Maquinari de desenvolupament	14
4 Decisions de disseny	15
4.1 Vulkan com a API gràfica	15
4.1.1 Problema i alternatives	15
4.1.2 Criteri i decisió	15
4.1.3 Cost de la decisió	16
4.2 C com a llenguatge	16

4.2.1	Problema i alternatives	16
4.2.2	Una precisió sobre el disseny orientat a dades	17
4.2.3	Criteri i decisió	17
4.2.4	Cost de la decisió	18
4.3	El motor com a biblioteca compartida i <i>framework</i>	18
4.3.1	Problema i alternatives	18
4.3.2	Criteri i decisió	18
4.3.3	Cost de la decisió	18
4.4	Gestió explícita de memòria	19
4.4.1	Problema i alternatives	19
4.4.2	La convergència de dues exigències	19
4.4.3	Criteri i decisió	19
4.4.4	Cost de la decisió	20
4.5	Separació entre la part independent de l'API i la implementació	20
4.5.1	Problema i alternatives	20
4.5.2	Criteri i decisió	20
4.5.3	Cost de la decisió	20
4.6	Model de recursos: referències, identificadors i generacions	21
4.6.1	Problema i alternatives	21
4.6.2	Criteri i decisió	21
4.6.3	Cost de la decisió	21
5	Disseny i implementació	23
5.1	Arquitectura general i cicle de vida	23
5.2	Gestió de memòria	23
5.3	Sincronització CPU/GPU i el bucle de <i>frame</i>	23
5.4	Sistemes de recursos	23
5.5	Resta de subsistemes	23
6	Resultats i discussió	24
6.1	Metodologia de mesura	24
6.2	Assoliment dels objectius funcionals	24
6.3	Experiments	24
6.4	Discussió	24
6.5	Limitacions de l'avaluació	24
7	Conclusions	25
7.1	Conclusions del treball realitzat	25
7.2	Punts forts i punts febles	25
7.3	Limitacions	25
7.4	Línies de continuació	25
	Referències	26
	Annexos	27

Índex de figures

Figura 1 Model en capes d'un motor de jocs, adaptat de [1], amb indicació de les capes cobertes per Hefest.	4
--	---

Índex de taules

Taula 1	Correspondència entre les capes del model [1] i l'estat del motor desenvolupat.	4
Taula 2	Correspondència entre els conceptes de Mantle i els de Vulkan.	7
Taula 3	Fases del desenvolupament del motor.	10
Taula 4	Distribució de la càrrega de treball per fases.	13
Taula 5	Equips emprats per al desenvolupament i les mesures.	14

Acrònims

[Capítol obligatori. Ordre alfabètic. Els acrònims que provenen de paraules en una altra llengua van en cursiva. Un cop desplegats aquí, ja no es tornen a desplegar en tot el document.]

API *Application Programming Interface*

DOD *Data-Oriented Design*

FPS *Frames Per Second*

GPU *Graphics Processing Unit*

SPIR-V *Standard Portable Intermediate Representation — Vulkan*

TFG *Treball de Final de Grau*

UBO *Uniform Buffer Object*

UMA *Unified Memory Architecture*

WSI *Window System Integration*

1 Introducció

1.1 Context i motivació

*[Per què un motor de jocs propi, i per què ara. Enllaçar amb la formació del grau: quins conceptes s'hi han tocat i a quin nivell. Aquí es justifica que el projecte és una **aprofundiment** en un tema del grau, que és literalment el que demana RA1.]*

1.2 Objectius

L'objectiu general d'aquest treball és dissenyar, implementar i avaluar un motor de jocs escrit en C amb un renderitzador basat en Vulkan, documentant-ne les decisions de disseny contra les fonts primàries que les fonamenten.

Aquest objectiu general es concreta en els objectius específics següents, cadascun dels quals té una resposta explícita al capítol de resultats:

1. Implementar una capa d'abstracció de plataforma que permeti executar el motor sobre Windows i Linux sense modificacions al codi dels subsistemes superiors.
2. Implementar un renderitzador sobre Vulkan capaç de carregar geometria, textures i materials des de disc i dibuixar-los amb una càmera controlable.
3. Dissenyar un model de gestió de memòria explícit, en què la ubicació i el cicle de vida de l'estat de cada subsistema siguin decisions del motor i no del sistema d'assignació subjacent.
4. Validar el funcionament del motor sobre arquitectures de memòria unificada i de memòria dedicada, i contrastar-ne el comportament.
5. Documentar cada decisió estructural del motor amb la font primària que la fonamenta, de manera que en sigui possible la justificació independent.
6. Avaluar empíricament un conjunt de decisions del renderitzador i quantificar l'impacte de cadascuna.

[Revisar aquesta llista quan el capítol 5 estigui tancat. Cada objectiu ha de tenir una secció al capítol 6 que hi respongui de manera comprovable; si algun no s'ha assolit, és preferible reformular-lo aquí que deixar-lo sense resposta. El rúbriic puntua directament que els objectius estiguin ben delimitats.]

1.3 Abast

El treball comprèn el desenvolupament de les capes fonamentals d'un motor de jocs i d'un renderitzador funcional, tal com s'ha delimitat a la Taula 1: capa de plataforma, sistemes bàsics, gestió de recursos, renderitzador i tractament de dispositius d'entrada. Inclou igualment la documentació tècnica del disseny i l'avaluació empírica del comportament del renderitzador.

Queden explícitament fora de l'abast els subsistemes següents, que un motor comercial inclouria però que no són requisit per als objectius plantejats:

Simulació física i detecció de col·lisions Constitueixen per si soles un domini d'una extensió comparable a la del renderitzador.

Àudio, animació i xarxa Són subsistemes funcionals que es recolzen sobre les capes fonamentals però que no en condicionen el disseny.

Sistema de tasques concurrents El motor és d'un sol fil d'execució. Es tracta de la limitació més rellevant del treball i se'n discuteixen les implicacions al capítol 7.

Eines d'edició i canonada d'actius No hi ha editor de nivells ni procés automatitzat de conversió d'actius més enllà de la compilació dels *shaders*.

Lògica de joc L'aplicació que acompanya el motor té l'única finalitat de demostrar-ne i exercitar-ne les capacitats; no constitueix un joc.

Aquesta delimitació respon al criteri exposat a la secció sobre motors existents: l'objectiu del treball és la comprensió de les capes que qualsevol motor necessita abans de poder dibuixar res, no l'assoliment de la paritat funcional amb un producte comercial.

1.4 Organització del document

[Un paràgraf per capítol. Ho exigeix la plantilla.]

2 Estat de l'art i marc tecnològic

2.1 Arquitectura d'un motor de jocs

Un motor de jocs no és un programa monolític, sinó un conjunt de subsistemes organitzats en capes, on les capes superiors depenen de les inferiors però no a la inversa [1]. Quan una capa inferior depèn d'una de superior es produeix una dependència circular, situació indesitjable en qualsevol sistema de programari perquè genera acoblament entre subsistemes, en dificulta la verificació i n'impedeix la reutilització.

Aquest principi no és una consideració merament teòrica. En un motor com el que es descriu en aquest treball, determina l'ordre en què els subsistemes s'han d'inicialitzar i, per tant, condiciona directament el disseny de la seqüència d'arrencada que es tracta al capítol 4.

2.1.1 Les capes d'un motor 3D típic

Un motor 3D complet es pot descompondre en setze capes [1], que al seu torn s'agrupen en quatre blocs segons la seva naturalesa.

El primer bloc no forma part del motor, sinó del seu fonament: el maquinari objectiu, els controladors de dispositiu, el sistema operatiu i els SDK de tercers i programari intermediari. Determinen les possibilitats i els límits de tot el que hi ha per sobre, però no els escriu qui desenvolupa el motor.

El segon bloc és el fonament del motor pròpiament dit. La capa d'independència de plataforma encapsula les diferències entre sistemes operatius i biblioteques natives, de manera que la resta del motor pugui ignorar-les. Els sistemes bàsics apleguen les utilitats transversals que tota aplicació gran necessita: assercions, gestió de memòria, biblioteca matemàtica, estructures de dades i registre d'esdeveniments. El gestor de recursos ofereix una interfície unificada per accedir als actius del joc, encara que cada motor el resol de manera diferent, des d'esquemes centralitzats fins a solucions *ad hoc* on el programador accedeix directament als fitxers [1].

El tercer bloc conté els subsistemes funcionals: el renderitzador, les eines de perfilatge i depuració, la detecció de col·lisions i la física, l'animació, els dispositius d'interfície humana, l'àudio i la xarxa i el multijugador en línia. Són els que donen al motor les seves capacitats visibles, i cadascun és, en si mateix, un sistema complex.

El quart bloc és el més proper al joc concret: els sistemes fonamentals de joc i els subsistemes específics del joc. És on la frontera entre motor i joc es difumina, i on cada projecte pren decisions pròpies.

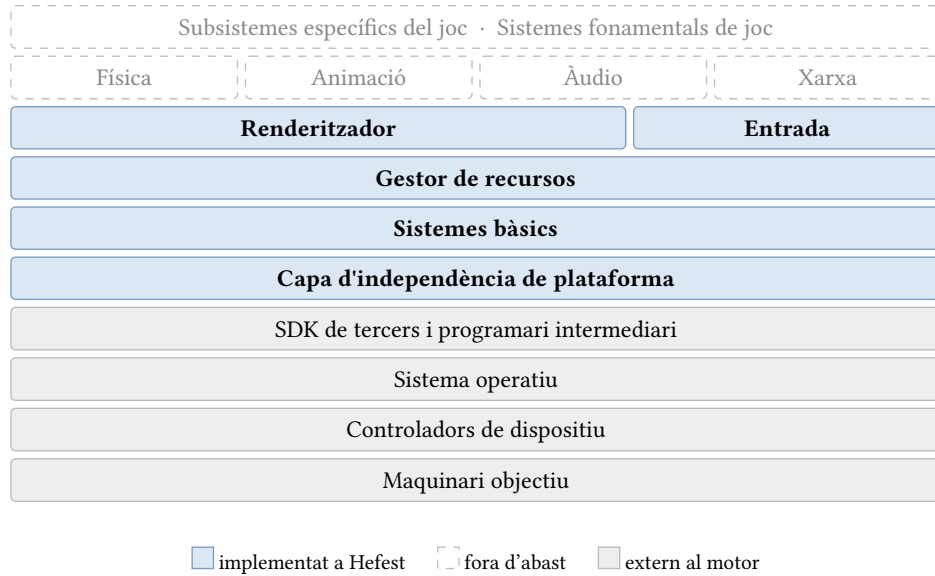


Figura 1: Model en capes d'un motor de jocs, adaptat de [1], amb indicació de les capes cobertes per Hefest.

2.1.2 Correspondència amb el motor desenvolupat

La Figura 1 situa el treball dins d'aquest model. Hefest implementa les tres capes fonamentals — independència de plataforma, sistemes bàsics i gestor de recursos— i dues de les funcionals: el renderitzador i els dispositius d'interfície humana. La resta queda explícitament fora de l'abast, tal com s'estableix a la secció corresponent del capítol 1.

Aquesta delimitació no és arbitrària. Les capes implementades són precisament les que qual-
sevol motor necessita abans de poder mostrar res per pantalla, i les que constitueixen els sistemes
de suport del motor [1]. Les capes omeses —física, animació, àudio, xarxa— són subsistemes
funcionals que es recolzen sobre les anteriors però que no en són requisit previ. Un motor sense
àudio continua sent un motor; un motor sense gestió de memòria ni capa de plataforma, no.

Taula 1: Correspondència entre les capes del model [1] i l'estat del motor desenvolupat.

Capa	Estat a Hefest	On es tracta
Independència de plataforma	Implementada (Win32, XCB/X11)	Cap. 4 i 5
Sistemes bàsics	Implementats	Cap. 5
Gestor de recursos	Parcial: textures i materials	Cap. 4 i 5
Renderitzador	Implementat sobre Vulkan	Cap. 4, 5 i 6
Perfilatge i depuració	Parcial: instrumentació pròpia	Cap. 5 i 6
Dispositius d'interfície humana	Implementada: teclat i ratolí	Cap. 5
Col·lisions i física	No implementada	Fora d'abast
Animació	No implementada	Fora d'abast
Àudio	No implementada	Fora d'abast
Xarxa i multijugador	No implementada	Fora d'abast
Sistemes fonamentals de joc	No implementats	Fora d'abast
Subsistemes específics del joc	Mínims: aplicació de proves	Cap. 5

Convé remarcar una asimetria que la Taula 1 no mostra: el fet que una capa estigui implementada no vol dir que ho estigui de manera completa. El gestor de recursos, per exemple, existeix en forma de dos sistemes especialitzats —textures i materials— sense la capa genèrica d'accés unificat als actius que el model preveu [1]. Aquesta mena de limitacions es documenten al lloc corresponent de cada subsistema i es recullen de manera conjunta a les conclusions.

[PENDENT: revisar la fila «Perfilatge i depuració» i la menció a la instrumentació quan el subsistema estigui implementat. Ara mateix només hi ha comptadors de memòria per etiqueta.]

2.2 De OpenGL a Vulkan: per què existeix Vulkan

Aquesta secció no és un excurs històric. La forma d'un motor de jocs modern està determinada en bona part per la naturalesa de l'API gràfica sobre la qual es construeix, i entendre per què Vulkan és com és equival a entendre per què el motor descrit en aquest treball ha de contenir un assignador de memòria, un model explícit de ritme de *frames* i un sistema de recursos propi.

2.2.1 El model clàssic i el seu cost

Durant més de dues dècades, la interacció entre aplicació i maquinari gràfic va seguir un model en què el controlador feia gran part de la feina difícil: la gestió dels riscos d'accés concurrent a la memòria, la sincronització entre processador i GPU, i l'assignació de memòria de vídeo [2]. Aquest repartiment de responsabilitats tenia virtuts considerables. El model era estable —codi escrit vint anys enrere continuava funcionant—, era senzill d'utilitzar, i va ser suficient per a una gamma molt àmplia d'aplicacions [2].

El problema és que aquesta comoditat té un preu mesurable. El sobrecost del controlador es tradueix en cicles de processador, ocupació de la memòria cau, consum energètic i, indirectament, rendiment de la GPU [2]. Per a aplicacions poc exigents el preu és irrellevant; per a jocs amb escenes complexes deixa de ser-ho. Els límits identificats del model clàssic són concrets: no escala amb la complexitat de l'escena, el seu model de fils d'execució és inadequat per a processadors multinucli, i l'abstracció del maquinari que proposa ja no es correspon amb l'arquitectura de les GPU modernes [2].

El diagnòstic que el mateix consorci Khronos en fa uns anys més tard és més detallat i val la pena enumerar-lo, perquè cada punt té una conseqüència directa sobre el disseny d'un motor [3]. El model de programació no es correspon amb el maquinari de la GPU —especialment en dispositius mòbils— i és el controlador qui n'amaga el desajust. El cost en processador prové sobretot de la validació d'estat i del seguiment de dependències que el controlador ha de fer. Els controladors resultants són complexos i difícils de predir, amb errors i camins ràpids diferents a cada GPU. I, de manera fonamental, el model és d'un sol fil d'execució, cosa que impedeix aprofitar els processadors multinucli.

2.2.2 Un primer intent des de dins: AZDO

La primera resposta a aquests límits no va ser substituir l'API, sinó intentar resoldre'ls dins del marc existent. Aquesta és exactament la pregunta que planteja la iniciativa AZDO —*Approaching Zero Driver Overhead*—: si els inconvenients del model clàssic es poden corregir sense renunciar-ne als avantatges [2].

La resposta tècnica consisteix a desacoblar el processador de la GPU i fer que es comuniquin a través de la memòria en lloc de fer-ho a través de crides a l'API. L'aplicació escriu a memòria les ordres de dibuix indirectes, els objectes de *buffer* i les referències a textures; la GPU llegeix les ordres directament de memòria; i la intervenció del controlador es redueix al mínim. Com

que la comunicació passa per memòria i no per l'API, diversos fils d'execució hi poden escriure simultàniament sense necessitat de cap mecanisme nou [2].

Els resultats reportats són d'entre cinc i quinze vegades el rendiment original en els casos limitats pel controlador [2]. Es tracta, doncs, d'una millora substancial obtinguda sense trencar res: AZDO destaca explícitament com a avantatges que la proposta no requereix un model d'objectes nou ni invalida les aplicacions existents [2].

Aquest darrer punt és el que fa que AZDO sigui rellevant per a entendre Vulkan. La direcció que assenyala —comunicar a través de memòria, minimitzar la intervenció del controlador, permetre el paral·lisme real— és precisament la que Vulkan acabarà adoptant. La diferència és que Vulkan la porta fins al final, i per fer-ho pren la decisió contrària respecte de la compatibilitat: sí que defineix un model d'objectes nou i sí que trenca amb les aplicacions existents.

2.2.3 Mantle: traslladar la responsabilitat a l'aplicació

El pas següent el marca Mantle, l'API que AMD va publicar el 2014 i que documenta el 2015 [4]. El diagnòstic de partida és el mateix: els models de programació de la generació anterior no són solucions idònies en escenaris on el desenvolupador necessita un control més estret del sistema gràfic i un sobrecost d'execució menor [4].

La solució proposada s'articula al voltant d'un principi que val la pena citar literalment, perquè és el que explica la forma de tot el que ve després:

L'aplicació és l'àrbitre del renderitzat correcte i l'únic dipositari de l'estat persistent. L'anàlisi de les API actuals indica que una solució eficient per a lots petits només es pot assolir quan el controlador és tan mancat d'estat com sigui possible.

— [4], traducció pròpia

D'aquest principi se'n deriva un segon, encara més explícit: quan la implementació genèrica d'una funcionalitat s'ha demostrat massa ineficient en altres API, la responsabilitat es trasllada a l'aplicació, perquè aquesta coneix millor el context de renderitzat i pot aplicar estratègies d'optimització més intel·ligents. L'exemple que se'n dona és precisament el que més afecta l'arquitectura d'un motor: la gestió de la memòria de vídeo passa a ser responsabilitat de l'aplicació [4].

El manifest per a desenvolupadors que tanca el capítol introductori de Mantle és inusualment franc sobre què implica això. El controlador deixa de proporcionar seguretat, millores de rendiment i solucions de compromís; no crea fils d'execució addicionals d'amagat de l'aplicació, no fa validació extensiva als camins crítics de rendiment i no recompila *shaders* en segon pla. A canvi, el desenvolupador ha d'assumir la validació, la correcció d'usos indeguts de l'API i la sincronització. El document ho formula sense embuts: aquesta responsabilitat addicional és el preu que s'ha de pagar per obtenir-ne els avantatges [4].

2.2.4 De Mantle a Vulkan

La correspondència entre els conceptes de Mantle i els de Vulkan és directa i es pot comprovar comparant l'estructura dels dos documents de referència, tal com recull la Taula 2.

Taula 2: Correspondència entre els conceptes de Mantle i els de Vulkan.

Concepte a Mantle [4]	Equivalent a Vulkan [5]
<i>GPU memory heaps</i> , objectes de memòria	VkMemoryHeap, VkDeviceMemory
Cues i <i>command buffers</i>	VkQueue, VkCommandBuffer
<i>Command buffer fences</i>	VkFence
<i>Queue semaphores</i>	VkSemaphore
<i>Events</i>	VkEvent
<i>Descriptor sets</i> i les seves actualitzacions	VkDescriptorSet
Imatges i vistes d'imatge	VkImage, VkImageView
Estats de recurs i preparació	Disposicions d'imatge i barreres
Estat estàtic i estat dinàmic	Estat de <i>pipeline</i> i estat dinàmic
<i>Graphics pipelines</i>	VkPipeline
Capa de validació	Capes de validació

La coincidència no es limita a la nomenclatura. La descripció del model d'execució és pràcticament la mateixa als dos documents: l'execució dels *command buffers* dins d'una cua és sèrie, però cues diferents poden executar-se de manera asíncrona, i és responsabilitat de l'aplicació sincronitzar-les; l'enviament a una cua retorna el control a l'aplicació abans que la feina s'executi [4]. La formulació de Vulkan és equivalent: les ordres d'enviament a cua haurien de retornar tan bon punt la feina s'ha enviat, sense esperar-ne la finalització, i no hi ha restriccions d'ordenació implícites entre operacions de cues diferents [5].

El mateix passa amb el model de memòria. Tots dos documents descriuen un dispositiu que exposa diversos *heaps* amb propietats diferents, adverteixen que no tota la memòria de la GPU és necessàriament accessible des del processador, i prescriuen que l'aplicació no faci cap suposició sobre aquesta visibilitat sinó que consulti les propietats que el sistema reporta [4], [5]. Mantle ja hi descriu, el 2015, el patró que se'n deriva: quan un *heap* concret no és accessible des del processador, les dades s'hi carreguen mitjançant operacions de còpia de GPU des d'un objecte de memòria que sí que ho és [4]. Aquesta advertència no és una formalitat, i el capítol 6 en mostra les conseqüències pràctiques sobre maquinari amb memòria unificada.

Aquesta correspondència no és casual. Les discussions de disseny de Vulkan comencen l'octubre de 2012 i s'acceleren entre el juliol i l'agost de 2014 amb dos fets decisius: el compromís de diversos desenvolupadors de programari clau i la cessió de Mantle per part d'AMD [3]. L'API resultant es publica el febrer de 2016, acompanyada de controladors conformes de quatre fabricants, un compilador de GLSL a SPIR-V i eines de depuració i validació [3].

2.2.5 El principi de control explícit

El fonament de Vulkan es formula com un pacte recíproc, i no com una simple transferència de càrrega cap a l'aplicació [3]. L'aplicació es compromet a dir al controlador què farà, amb prou detall perquè no ho hagi d'endevinar i en el moment en què el controlador necessita saber-ho. A canvi, el controlador es compromet a fer exactament allò que se li ha demanat, quan se li ha demanat, i molt de pressa.

El contrast amb el model anterior és explícit. Les API clàssiques permeten especificar informació important molt tard i canviar-la en qualsevol moment, la qual cosa és còmoda però té un

cost de rendiment elevat; i els seus controladors sovint ajornen feina, la traslladen a un altre fil o fins i tot ignoren ordres basant-se en suposicions sobre la intenció de l'aplicació [3]. Els controladors de Vulkan no ho fan.

D'aquest pacte se'n deriva la política de gestió d'errors, que és igualment explícita: Vulkan està optimitzat per a aplicacions correctes i, en general, no comprova usos indeguts, no fa seguiment de dependències i no proporciona seguretat entre fils; incomplir-ne les regles produeix comportament indefinit [3]. Sí que informa dels errors que l'aplicació no pot preveure —manca de memòria, pèrdua del dispositiu— i deixa la resta de la comprovació a les capes de validació, pensades per activar-se durant el desenvolupament. Aquesta divisió és la que justifica que el motor descrit en aquest treball habiliti la validació únicament a les compilacions de depuració, tal com es detalla al capítol 4.

2.2.6 Conseqüència: l'API imposa l'arquitectura

Convé tornar a la llista de responsabilitats que el model clàssic assignava al controlador: gestió de riscos d'accés, sincronització i assignació de memòria [2]. Vulkan les trasllada totes tres a l'aplicació. Aquest trasllat no és un detall d'implementació, sinó el fet que determina quins subsistemes ha de tenir qualsevol motor construït sobre aquesta API:

Assignació de memòria si el controlador no gestiona la memòria del dispositiu, el motor necessita una estratègia pròpia d'assignació i subassignació, i ha de decidir com transfereix les dades cap a memòria no visible des del processador.

Sincronització si no hi ha ordenació implícita entre operacions, el motor ha de construir explícitament el seu model de ritme de *frames* amb primitives de sincronització, i ha de decidir quantes imatges processa simultàniament.

Estat i validació si el controlador és mancat d'estat i no valida als camins crítics, el motor ha de decidir on col·loca l'estat de renderitzat i com n'obté diagnòstic durant el desenvolupament sense pagar-ne el cost en producció.

Aquests tres punts són, precisament, l'índex del capítol 4. Les decisions de disseny que s'hi documenten no responen a preferències estètiques sinó a obligacions que l'API imposa, i és per això que aquesta secció ocupa el lloc que ocupa dins de l'estat de l'art.

2.3 Motors existents i per què construir-ne un

El panorama actual dels motors de propòsit general està dominat per unes poques solucions madures. Unity [6] i Unreal Engine [7] ocupen la major part del mercat comercial, mentre que Godot [8] s'ha consolidat com l'alternativa de codi obert més estesa. Totes tres ofereixen editors visuals, sistemes de física, àudio, animació i interfície d'usuari, exportació a múltiples plataformes i comunitats extenses de documentació i recursos.

Davant d'aquesta oferta, construir un motor propi no té sentit com a proposta de producte. Cap projecte individual pot competir amb l'abast d'aquestes eines, i per a desenvolupar un joc la decisió racional és gairebé sempre fer servir-ne una. La justificació d'aquest treball és d'una altra naturalesa: allò que aquests motors aporten com a valor —abstreure la complexitat del maquinari gràfic, de la gestió de memòria i de la sincronització— és precisament allò que aquí constitueix l'objecte d'estudi. Un motor comercial resol aquests problemes i n'amaga la solució; l'objectiu d'aquest projecte és entendre-los.

2.3.1 El llinatge com a norma en el desenvolupament de motors

Convé situar aquesta decisió en el context de com es construeixen realment els motors de jocs, perquè la imatge del motor escrit des de zero és poc representativa de la pràctica del sector. Els motors deriven habitualment d'altres motors. La tecnologia de Quake, per exemple, va donar lloc a una successió de títols i motors que passa per Sin, F.A.K.K. 2 i els diversos Medal of Honor, i fins i tot el motor Source de Valve en té arrels llunyanes [1].

Aquesta pràctica té una dimensió pedagògica reconeguda. El codi font de Quake i Quake II és lliurement accessible i, tot i estar escrit íntegrament en C i haver quedat desfasat, es considera un exemple net i ben estructurat de com es construeixen motors de nivell industrial; la recomanació explícita és descarregar-lo, compilar-lo i analitzar-ne el funcionament executant-lo pas a pas sota un depurador [1].

Estudiar una implementació existent i construir-hi a sobre no és, doncs, una drecera respecte del mètode habitual, sinó el mètode habitual. Aquest treball adopta aquesta aproximació de manera deliberada i explícita, i el capítol 3 en detalla la metodologia i les fonts.

2.4 Projectes i materials de referència

El desenvolupament d'aquest motor s'ha recolzat en un conjunt de projectes de codi obert i materials formatius que aborden problemes equivalents. Se'n detallen aquí els que han tingut un pes real, per ordre d'importància.

Kohi Sèrie de desenvolupament d'un motor de jocs en C i Vulkan, publicada com a llista de reproducció de vídeos [9] amb el codi font associat disponible públicament [10]. És la implementació de referència principal del projecte: n'estableix l'arquitectura general, l'ordre d'incorporació dels subsistemes i bona part de les convencions de nomenclatura. El capítol 3 en documenta l'ús concret i delimita què s'ha pres com a base i què constitueix aportació pròpia.

Handmade Hero Sèrie que desenvolupa un joc complet en C des de zero, sense biblioteques externes [11]. Rellevant sobretot pel tractament de la gestió de memòria basada en regions i per l'argumentació sobre control explícit dels recursos.

bgfx Biblioteca de renderitzat multiplataforma que abstruï diverses API gràfiques rere una interfície comuna [12]. Serveix com a referència de com es dissenya la frontera entre la part del renderitzador independent de l'API i la part específica de cada implementació, qüestió que es tracta al capítol 4.

Exemples de Vulkan de Sascha Willems Col·lecció extensa d'exemples que cobreix la major part de les funcionalitats de l'API [13]. Emprada com a referència puntual de contrast en la implementació de tècniques concretes.

A aquests projectes cal afegir-hi les dues obres que han servit de fonament teòric al llarg de tot el treball: *Game Engine Architecture* [1], per a l'arquitectura de motors, i l'especificació de Vulkan [5], com a font normativa per a tot el que afecta l'API gràfica.

3 Metodologia i planificació

3.1 Metodologia de desenvolupament

El desenvolupament s'ha organitzat de manera incremental per subsistemes. Cada subsistema s'ha portat fins a un estat funcional, s'ha integrat amb els anteriors i se n'ha documentat el disseny abans de començar el següent. El resultat és un historial de 38 *commits* repartits entre l'agost de 2025 i el juny de 2026, en què cada un correspon a una unitat de treball coherent i identificable.

L'ordre d'aquesta incorporació no és arbitrari. L'estructura en capes descrita a la Figura 1 imposa un ordre de dependències que en determina bona part: no es pot construir el renderitzador sense una capa de plataforma que proporcioni una finestra, ni el sistema de textures sense un renderitzador capaç de crear recursos a la GPU. Dins d'aquesta restricció s'ha prioritzat sempre arribar com abans millor a un estat executable, encara que fos mínim, per disposar de verificació empírica contínua.

3.1.1 Fases del desenvolupament

Retrospectivament, el treball s'agrupa en tres fases amb caràcter metodològic diferenciat, recollides a la Taula 3.

Taula 3: Fases del desenvolupament del motor.

Fase	Període	Contingut
Posada en marxa	ago. – set. 2025	Registre i assercions, capa de plataforma, capa d'aplicació, memòria, esdeveniments, entrada, arrencada de Vulkan fins a esborrar la pantalla, biblioteca matemàtica.
Refactorització	des. 2025	Revisió de contenidors i tipus booleans, projecte de proves unitàries, assignador lineal, adopció del patró d'inicialització en dues crides i migració de tots els subsistemes existents.
Consolidació	des. 2025 – jun. 2026	Sistema de fitxers, canonada gràfica completa, uniformes i descriptors, constants d'inserció, textures des de disc, sistema de textures, suport per a Linux, documentació i adaptació a GPU integrades.

La fase de refactorització mereix un comentari, perquè il·lustra com ha funcionat realment el procés. El patró d'inicialització en dues crides que caracteritza tots els subsistemes del motor no es va dissenyar per endavant: va aparèixer en aquesta fase, quan ja existien set subsistemes amb esquemes d'arrencada heterogenis, i es va aplicar retroactivament a tots. El mateix va passar amb l'assignador lineal, que es va introduir simultàniament perquè el patró nou el necessitava. L'arquitectura, doncs, no estava completament planificada des de l'inici; una part es va establir només després de tenir prou subsistemes com per veure què tenien en comú.

3.1.2 Verificació

La verificació s'ha fet per tres vies complementàries, cadascuna amb un abast diferent.

La primera és l'execució de l'aplicació de proves, que ha funcionat com a comprovació contínua d'integració: qualsevol regressió en el camí de renderitzat es manifesta immediatament de manera visible.

La segona són les capes de validació de Vulkan, que constitueixen l'eina principal de correcció per a tot el codi gràfic. Com s'ha vist a la secció corresponent del capítol 2, l'API no comprova els usos indeguts als camins crítics, de manera que aquestes capes són l'únic mecanisme sistemàtic de detecció d'errors d'ús durant el desenvolupament.

La tercera són les proves unitàries, incorporades a la fase de refactorització juntament amb un projecte de proves independent. La seva cobertura és limitada —únicament l'assignador lineal i la taula de dispersió— perquè s'han aplicat només als components amb comportament aïllable i verificable sense context gràfic. Aquesta limitació es discuteix al capítol 7.

3.1.3 Documentació com a part del mètode

Paral·lelament al codi s'ha mantingut un corpus de documentació tècnica de 32 documents i aproximadament 1.800 línies, amb un document per subsistema i una estructura fixa: propòsit, fitxers, interfície pública, funcionament, decisions de disseny i justificació, limitacions conegudes i referències creuades.

Aquesta documentació no és un subproducte redactat al final, sinó una part del mètode de treball, i respon a un objectiu concret: obligar a explicitar la justificació de cada decisió en el moment de prendre-la, i registrar-ne les limitacions mentre encara són presents. Bona part del material dels capítols 4 i 7 en prové directament. La secció de limitacions conegudes de cada document ha resultat especialment útil, perquè documentar allò que un subsistema no fa bé és el que permet, més endavant, distingir una decisió deliberada d'un descuit.

3.2 Procedència de les fonts i ús de projectes de referència

Tal com s'ha exposat a la secció sobre el llinatge en el desenvolupament de motors, construir sobre una implementació existent és la pràctica habitual del sector i una via d'aprenentatge reconeguda [1]. Aquest treball hi recorre de manera deliberada, i aquesta secció en detalla l'abast amb precisió.

3.2.1 Base emprada

La implementació de referència principal ha estat la sèrie Kohi [9], amb el codi font associat [10]. Se n'han pres l'arquitectura general del motor, l'ordre d'incorporació dels subsistemes i les convencions de nomenclatura, que en el codi d'aquest projecte apareixen amb el prefix *h* en lloc del prefix original.

La sèrie s'ha fet servir com a implementació de referència, no com a transcripció. El criteri de treball ha estat que cada decisió estructural s'estudiés de manera independent contra les fonts primàries —*Game Engine Architecture* [1] per a les qüestions d'arquitectura, i l'especificació de Vulkan [5] per a tot el que afecta l'API gràfica— abans de documentar-la. El capítol 4 recull el resultat d'aquest exercici, i l'Annex A en conté la correspondència detallada entre cada decisió i la font que la fonamenta.

3.2.2 Aportacions pròpies

Els elements següents no provenen de la implementació de referència i són verificables a l'historial del repositori.

Adaptació a GPU integrades El *commit* `a0f9a24` adapta el codi de Vulkan a dispositius no discrets i corregeix un defecte de sincronització subjacent que ho impedia. El defecte només

es manifesta quan el nombre d'imatges de la *swapchain* i el nombre de *frames* en vol difereixen, cosa que no succeeix al maquinari sobre el qual es desenvolupa la sèrie de referència. Localitzar-lo va requerir entendre el model de sincronització de l'API, no reproduir-lo. El capítol 5 en detalla el mecanisme.

Portabilitat i conformitat El *commit* 317f497 elimina els vectors de longitud variable del codi i resol diversos problemes que impediien l'execució sobre Linux.

Corpus de documentació tècnica Els 32 documents descrits a la secció anterior, amb la justificació i les limitacions de cada subsistema.

Anàlisi de correspondència amb les fonts primàries El document que relaciona cada decisió de disseny del motor amb la secció corresponent de les fonts normatives, recollit a l'Annex A.

Avaluació empírica El disseny experimental, la instrumentació i les mesures del capítol 6, que no formen part de la sèrie de referència.

[Actualitzar aquesta llista a mesura que s'implementin l'assignador de frame i la capa d'instrumentació, tots dos derivats directament de [1] i no de la implementació de referència.]

3.2.3 Delimitació

Convé ser explícit sobre el revers d'aquesta llista: la major part dels subsistemes del motor segueixen l'estructura de la implementació de referència, i el treball propi en aquests casos consisteix a haver-los comprès, integrats, depurats i documentats, no a haver-los concebut. Aquesta distinció es manté al llarg del capítol 5, on cada subsistema indica quina és la seva procedència.

3.3 Planificació temporal

3.3.1 Premissa de disponibilitat

La planificació d'aquest treball es va dissenyar sobre una restricció coneguda des de l'inici: el projecte s'ha desenvolupat de manera simultània a una activitat laboral a jornada completa, essent el TFG l'única assignatura matriculada del curs. Aquesta circumstància no permet una dedicació uniforme al llarg de l'any acadèmic, i la planificació ho va incorporar com a premissa en lloc de tractar-ho com una contingència.

L'estratègia adoptada va ser concentrar l'esforç als períodes d'alta disponibilitat i mantenir una cadència reduïda però contínua durant la resta del temps. En conseqüència, una part substancial del desenvolupament —tota la fase de posada en marxa descrita a la Taula 3— es va dur a terme durant l'estiu de 2025, i les fases posteriors es van planificar amb una càrrega setmanal compatible amb la jornada laboral.

3.3.2 Pressupost d'hores

El treball té assignats 16 crèdits ECTS, que constitueixen l'ancoratge del pressupost de dedicació. *[Completar amb la ràtio d'hores per crèdit que estableix el pla d'estudis —habitualment entre 25 i 30— i el total resultant, que se situaria entre 400 i 480 hores. Verificar-ho a la normativa del grau en lloc d'estimar-ho.]*

Aquest total s'ha distribuït entre les fases de desenvolupament, l'avaluació i la redacció de la memòria segons la Taula 4.

[Completar la Taula 4 amb les hores previstes i reals per fase. El rúblic valora que s'hagin «definit tasques, valorant-ne les càrregues i complert la temporització»; amb la premissa de disponibilitat declarada més amunt, una distribució desigual entre fases és coherent amb el pla i no constitueix una desviació.]

3.3.3 Fases, càrrega i temporització

Taula 4: Distribució de la càrrega de treball per fases.

Fase	Període	Hores previstes	Hores reals
Posada en marxa	ago. – set. 2025	[-]	[-]
Refactorització	des. 2025	[-]	[-]
Consolidació	des. 2025 – jun. 2026	[-]	[-]
Avaluació i mesures	[-]	[-]	[-]
Redacció de la memòria	[-]	[-]	[-]
Total		[-]	[-]

[Afegir aquí el diagrama de Gantt. L'Annex E n'ha de contenir la versió detallada; al cos hi va la vista resumida per fases.]

3.3.4 Desviacions

[Secció breu i honesta. Reconèixer una desviació i explicar-ne la causa puntua més que presentar una planificació sense incidències que l'historial del repositori contradiu. Candidats a esmentar:

- 1. El sistema de materials es va endarrerir respecte de la previsió inicial.*
- 2. L'adaptació a GPU integrades no estava planificada; va sorgir en detectar que el motor no s'executava correctament sobre el maquinari de desenvolupament, i va consumir temps no previst. Va acabar sent una de les aportacions pròpies del treball, de manera que la desviació té un resultat defensable.*

]

3.4 Eines i entorn de desenvolupament

3.4.1 Llenguatge, compilació i construcció

El motor està escrit en C i es compila exclusivament amb Clang, en la versió 22.1.8 sobre l'entorn de desenvolupament descrit més avall. La construcció es gestiona amb GNU Make 4.4.1 mitjançant fitxers de construcció separats per component i per sistema operatiu, de manera que el motor, l'aplicació de proves i el projecte de proves unitàries es compilen de manera independent. El motor es genera com a biblioteca compartida i l'aplicació de proves s'hi enllaça.

3.4.2 API gràfica i eines associades

El renderitzador es construeix sobre Vulkan. Els *shaders* s'escriuen en GLSL i es compilen a SPIR-V amb `glslc` com a pas posterior a la construcció, atès que l'API només accepta aquest format intermedi [5]. La correcció de l'ús de l'API es verifica amb les capes de validació de Khronos, habilitades únicament a les compilacions de depuració.

Les eines de Vulkan emprades corresponen a la versió 1.4.357.

[Aclarir si es tracta del SDK de LunarG o dels paquets de la distribució. En l'entorn Linux, `glslc` reporta la versió 2026.3 (paquet 1:1.4.357.0) i la variable `VULKAN_SDK` no consta definida, cosa que

suggereix paquets del sistema; el fitxer README del projecte, en canvi, la indica com a requisit. Convé deixar el requisit documentat de manera coherent entre la memòria i el README.

Nota sobre les versions, que són tres i cal no barrejar-les: les eines són la 1.4.357; l'especificació citada com a font normativa és la 1.4.361; i la versió d'API que reporta el dispositiu en temps d'execució depèn del controlador —1.4.354 a la màquina amb GPU integrada—. Aquesta última és una dada de mesura i el seu lloc és el capítol 6, no aquí.]

3.4.3 Control de versions i documentació

El control de versions s'ha fet amb Git 2.55.0, amb un historial de *commits* d'unitat funcional que és, alhora, el registre cronològic del desenvolupament. La documentació tècnica del projecte es manté en format Markdown dins del mateix repositori, i aquesta memòria s'ha redactat amb Typst 0.15.1. L'anàlisi del codi durant el desenvolupament s'ha recolzat en clangd.

3.4.4 Maquinari de desenvolupament

L'entorn de desenvolupament i de mesura es detalla a la Taula 5. Les seves característiques no són un detall administratiu: el fet que la GPU sigui integrada i comparteixi memòria amb el sistema condiciona directament una part de les decisions del capítol 4 i és la premissa dels experiments del capítol 6.

Taula 5: Equips emprats per al desenvolupament i les mesures.

	Equip A — GPU integrada	Equip B — GPU discreta
Processador	Intel Core i7-1255U (12a gen.), 10 nuclis / 12 fils	AMD Ryzen <i>[model]</i>
GPU	Intel Iris Xe Graphics (ADL GT2), integrada	NVIDIA GeForce GTX 1650 Ti, dedicada
Arquitectura de memòria	Unificada amb el sistema	Memòria de vídeo dedicada
Controlador	Mesa 26.2.1 (codi obert d'Intel)	<i>[Controlador i versió]</i>
Memòria del sistema	16 GB	16 GB
Sistema operatiu	<i>[Distribució i nucli]</i>	<i>[Sistema i versió]</i>

La disponibilitat de dos equips amb arquitectures de memòria diferents no és una circumstància accessòria, sinó una condició que el treball aprofita de manera deliberada. L'equip A reporta el dispositiu com a `PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU`: no disposa de memòria local dedicada, i els tipus de memòria visibles des del processador coincideixen en bona part amb els locals al dispositiu. L'equip B, amb GPU dedicada, presenta la separació clàssica entre les dues.

Aquesta diferència travessa tot el treball. És la que va fer aflorar el defecte de sincronització documentat a la secció d'aportacions pròpies, que només es manifesta sobre l'equip A; és la que dona sentit a l'advertència de la documentació de Mantle sobre no presuposar la visibilitat directa de la memòria [4]; i és la que permet que els experiments del capítol 6 siguin comparatius en lloc de descriptius, contrastant el comportament de les dues arquitectures en lloc de caracteritzar-ne una de sola.

4 Decisions de disseny

Les decisions que recull aquest capítol no es presenten com a solucions òptimes. En programari rarament n'hi ha cap que ho sigui en termes absoluts: hi ha opcions que encaixen millor amb uns objectius determinats i pitjor amb uns altres, i el que fa útil una decisió és haver-la presa sabent amb quins objectius havia d'encaixar. El criteri que ha guiat el projecte ha estat, doncs, escollir en funció dels objectius plantejats al capítol 1 —i, en algun cas, de l'interès per entendre un mecanisme concret— després d'haver considerat les alternatives disponibles.

Cada secció segueix la mateixa estructura: quin problema planteja la decisió, quines alternatives es van considerar, quin criteri es va aplicar i què s'hi va guanyar i què s'hi va perdre. L'últim punt hi consta sempre, perquè és el que permet jutjar si el criteri era encertat.

4.1 Vulkan com a API gràfica

4.1.1 Problema i alternatives

La primera decisió del projecte condiciona la resta: sobre quina API gràfica es construeix el renderitzador. El criteri previ que va delimitar el conjunt d'alternatives va ser que es tractés d'una API amb ús professional real a la indústria del videojoc, atès que un dels propòsits del treball és entendre les eines que s'hi fan servir. Aquest criteri va deixar fora opcions com WebGPU, que no es van arribar a considerar.

Van quedar, per tant, tres candidates: OpenGL, Vulkan i DirectX 12.

OpenGL era la més coneguda de partida, atès que s'ha tractat durant el grau. Ara bé, el seu ús en desenvolupament professional de videojocs ha anat disminuint, i —cosa més determinant per a aquest projecte— el seu model clàssic delega al controlador la gestió de riscos d'accés, la sincronització i l'assignació de memòria [2], que són precisament els mecanismes que el treball es proposa entendre. Una API que els resol de manera transparent en fa impossible l'estudi.

Descartada OpenGL, la tria quedava entre les dues API de control explícit d'ús generalitzat: Vulkan i DirectX 12.

4.1.2 Criteri i decisió

Convé ser explícit sobre un punt: qualsevol de les dues hauria servit. L'objectiu plantejat no és dominar una API concreta sinó entendre com funciona una API gràfica moderna de control explícit, i tant Vulkan com DirectX 12 responen a aquesta descripció i tenen ús professional consolidat. La decisió, doncs, no es va prendre sobre la superioritat tècnica d'una respecte de l'altra.

El factor que la va determinar va ser la flexibilitat quant al sistema operatiu. DirectX 12 està limitat a plataformes de Microsoft, mentre que Vulkan permet executar el mateix codi sobre Windows i Linux, cosa que encaixava amb l'objectiu de portabilitat plantejat al capítol 1.

Aquesta previsió es va acabar exercint de manera efectiva, i no només com a possibilitat teòrica. El projecte va començar sobre Windows —la capa de plataforma inicial és la d'aquest sistema— i posteriorment es va portar a Linux, que ha estat l'entorn de desenvolupament principal durant els darrers mesos. Aquest canvi d'entorn, que respon a circumstàncies alienes al disseny del motor, no hauria estat possible amb DirectX 12. És, per tant, un cas en què el criteri aplicat va resultar rellevant per raons que en el moment de decidir només es podien anticipar de manera genèrica.

La portabilitat que aquesta decisió proporciona no s'esgota, a més, en els dos sistemes operatius emprats. Vulkan és igualment accessible sobre les plataformes d'Apple, si bé cal precisar per quina via: macOS no dona suport natiu a Vulkan —el seu entorn gràfic propi és Metal—, sinó que aquest suport s'obté mitjançant MoltenVK, una implementació que desplega un subconjunt gairebé complet de Vulkan per sobre de Metal i que tradueix els *shaders* de SPIR-V al llenguatge d'ombreig de Metal [14]. Aquesta implementació dona suport a la versió 1.4 de l'API, que és la que el motor sol·licita. Incorporar macOS requeriria, per tant, escriure la capa de plataforma corresponent i la creació de superfície específica, però no afectaria la resta del renderitzador. Queda fora de l'abast del treball, tal com s'ha delimitat al capítol 1.

Aquesta manera d'entendre la decisió té reflex, també, en l'arquitectura del motor. El renderitzador se separa en una part independent de l'API i una implementació concreta, i el tipus enumerat que identifica la implementació activa preveu, a més de Vulkan, els casos d'OpenGL i DirectX. Incorporar una segona implementació requeriria escriure-la i afegir una branca a la funció que en fa la selecció, sense modificar ni la part independent de l'API ni el codi del joc. Aquesta separació es descriu en detall més endavant en aquest mateix capítol.

Convé no exagerar l'abast de cap d'aquestes dues afirmacions. Ni existeix una implementació per a una API alternativa ni una capa de plataforma per a macOS, i escriure qualsevol de les dues no seria un exercici menor; en el cas de macOS, caldria a més verificar quines restriccions imposa un subconjunt de portabilitat sobre les funcionalitats concretes que el motor utilitza. El que la decisió preserva no és la portabilitat immediata, sinó el fet que triar Vulkan no hagi tancat cap d'aquestes portes.

4.1.3 Cost de la decisió

El cost principal no prové de triar Vulkan en lloc de DirectX 12 —on hauria estat comparable— sinó d'haver optat per una API de control explícit en lloc d'una de clàssica, i convé enunciar-lo sense atenuants. El mateix consorci Khronos reconeix que l'API és verbosa i complexa, que exposa nombroses arestes i que hi ha molt per aprendre [3]. La conseqüència pràctica és que la quantitat de codi necessària per arribar a dibuixar el primer triangle és d'un ordre de magnitud superior a la que requeriria OpenGL, i que una part substancial del calendari descrit al capítol 3 es va destinar a construir infraestructura que no produeix cap resultat visible.

A canvi, l'API ofereix un sobrecost de controlador molt inferior, repartible entre múltiples fils d'execució, i un rendiment més previsible [3]. D'aquests avantatges, aquest treball només n'aprofita parcialment el primer i el tercer: el motor és d'un sol fil, de manera que la capacitat de repartir la feina entre nuclis —que és probablement el guany estructural més important de l'API— queda sense explotar. Aquesta limitació es discuteix al capítol 7.

4.2 C com a llenguatge

4.2.1 Problema i alternatives

L'elecció del llenguatge es va plantejar entre C i C++, que són les dues opcions realistes per a un motor que ha d'interoperar amb una API en C i mantenir control sobre la disposició de la memòria.

C++ ofereix genèrics, gestió automàtica de recursos mitjançant el patró de concessió de recursos en la inicialització, sobrecàrrega d'operadors i una biblioteca estàndard extensa [15]. Aquest patró, en particular, permet adquirir recursos en el constructor i alliberar-los en el destructor, cosa que elimina les operacions d'assignació explícites del codi general [15].

4.2.2 Una precisió sobre el disseny orientat a dades

L'interès per les tècniques de disseny orientat a dades va motivar inicialment la inclinació cap a C. Convé, però, precisar l'abast d'aquest argument, perquè formulat de manera descuidada no se sosté.

El disseny orientat a dades és la pràctica de dissenyar programari desenvolupant transformacions sobre dades ben formades, on el criteri del que és ben format ve determinat pel maquinari objectiu i pels patrons de transformació que s'hi han d'aplicar [16]. És, per tant, una qüestió de disposició de les dades i de coneixement del maquinari, i no d'elecció de llenguatge: la font de referència afirma explícitament que no està arrelat en un sol llenguatge [16].

Aquesta mateixa font assenyala que el llenguatge que més se'n beneficia és C++. La construcció de la frase és, però, paral·lela a la que fa servir per al maquinari, on sosté que el que més se'n beneficia és aquell que presenta colls d'ampolla desequilibrats [16]; és a dir, aquell on el problema és més acusat i, en conseqüència, el marge de millora més gran. Llegida en aquest sentit, l'afirmació no diu que C++ sigui el vehicle més adequat per al disseny orientat a dades, sinó que és on hi ha més a guanyar-hi.

Aquesta lectura és coherent amb l'objecte real de la crítica que l'obra desenvolupa, que no és cap llenguatge concret sinó el disseny orientat a objectes com a metodologia: el retret central és que acobla el domini del problema amb la implementació, i que aquest acoblament genera una inèrcia que dificulta l'adaptació quan les dades canvien [16]. C++ no obliga a programar així, i el disseny orientat a dades hi és plenament practicable.

Cal admetre, tanmateix, un matis que la distinció anterior no cobreix del tot: si bé el llenguatge no imposa aquell estil, sí que en facilita l'adopció. Els constructors i destructors que s'executen implícitament en entrar i sortir d'àmbit, el despatx virtual i els contenidors genèrics d'objectes fan que l'organització que la crítica desaconsella sigui també el camí de mínima resistència. La distinció útil, doncs, no és entre llenguatges que permeten el disseny orientat a dades i llenguatges que no, sinó entre llenguatges que hi empenyen i llenguatges que empenyen cap a una altra banda.

En qualsevol cas, aquest treball no fa servir el disseny orientat a dades com a argument per triar C. El que sí que va aportar l'interès per aquestes tècniques va ser orientar l'atenció cap a la disposició explícita de les dades en memòria, i és aquesta preocupació —no la metodologia en si— la que té conseqüències sobre l'elecció del llenguatge.

4.2.3 Criteri i decisió

El criteri efectiu és l'explicitud. C no disposa de constructors ni destructors implícits, ni de gestió automàtica de recursos, ni d'assignacions ocultes rere abstraccions. Tota reserva de memòria, tot alliberament i tota còpia són visibles al lloc on succeeixen, i el llenguatge no ofereix cap mecanisme per delegar-ne la responsabilitat. Programar-hi obliga, per tant, a un tracte deliberat amb la memòria en cada punt on s'hi accedeix.

En un motor construït sobre una API que trasllada deliberadament a l'aplicació la gestió de memòria, la sincronització i el cicle de vida dels recursos —tal com s'ha exposat al capítol 2—, aquesta correspondència entre el model del llenguatge i el model de l'API té valor pràctic: allò que cal fer explícit a la GPU també ho és al codi que la governa. En un treball on la disposició i el cicle de vida de les dades constitueixen l'objecte d'estudi, un llenguatge que els gestioni en nom del programador n'obstaculitza l'anàlisi, encara que permetés obtenir el mateix resultat amb menys codi.

Convé precisar l'ordre en què es van prendre aquestes decisions, atès que el capítol 3 documenta l'ús d'una implementació de referència també escrita en C. L'elecció del llenguatge va ser prèvia: la cerca de materials de referència es va fer un cop presa, i el fet que estiguessin escrits en C va ser un criteri de selecció d'aquests materials i no un factor que intervingués en la decisió.

4.2.4 Cost de la decisió

L'absència de genèrics obliga a implementar els contenidors mitjançant macros i aritmètica de mides en temps d'execució, amb la pèrdua de comprovació de tipus que això comporta. La gestió manual de recursos trasllada al programador la responsabilitat que el patró de concessió de recursos en la inicialització automatitza, amb el risc de fuites i alliberaments duplicats que se'n deriva. I la biblioteca estàndard, molt més reduïda, obliga a implementar funcionalitat que en C++ seria immediata, com ara la manipulació de cadenes o les estructures de dades associatives.

Aquests costos són reals i el motor els paga. La decisió es defensa perquè l'objectiu del treball no és minimitzar l'esforç d'implementació, sinó fer visible el comportament que un motor de jocs ha de controlar.

4.3 El motor com a biblioteca compartida i *framework*

4.3.1 Problema i alternatives

Un motor i el joc que s'hi construeix han de repartir-se el control del flux d'execució, i cal decidir quina de les dues parts el posseeix. Hi ha dues maneres habituals de resoldre-ho [1]. Una biblioteca és un conjunt de funcions i classes que l'aplicació invoca com li convé: ofereix màxima flexibilitat al programador, a canvi que aquest hagi d'entendre com utilitzar-la correctament. Un *framework*, en canvi, és una aplicació parcialment construïda en què el bucle principal ja està escrit però buit, i que el programador completa proporcionant les implementacions que hi manquen; en aquest cas té poc o cap control sobre el flux general.

4.3.2 Criteri i decisió

El motor s'ha construït com a *framework*. La funció `main` resideix dins de la biblioteca del motor i no al costat del joc, i és el motor qui condueix la seqüència d'arrencada, el bucle principal i el tancament.

El joc, per la seva banda, ha de proporcionar una funció que construeixi la seva instància i n'ompli una estructura amb quatre punters a funció —inicialització, actualització, renderitzat i resposta a canvis de mida— juntament amb la configuració de l'aplicació i un punter a l'estat propi del joc, del qual el motor no coneix l'estructura interna. Abans de continuar, el punt d'entrada verifica que els quatre punters existeixin i avorta amb un diagnòstic si no és així.

El criteri que va portar a aquesta decisió és que el bucle principal conté ordenacions que no són negociables i que no convé que cada joc hagi de tornar a resoldre. El capítol 5 en detalla un cas concret: l'actualització de l'estat d'entrada ha de ser l'última operació de cada iteració, perquè és la que converteix l'estat actual en estat previ i permet consultar transicions de tecla a la iteració següent. Fer-ho en un altre punt no produeix cap error visible, però trenca silenciosament la detecció de polsacions. Ordenacions d'aquesta mena justifiquen que el bucle sigui responsabilitat del motor.

4.3.3 Cost de la decisió

El cost és el que s'atribueix habitualment a aquest enfocament: el joc perd el control del flux [1]. Un joc que necessités una estructura de bucle diferent —per exemple, per integrar-se dins d'una

aplicació amfitriona amb el seu propi bucle d'esdeveniments— no podria fer-ho sense modificar el motor.

Hi ha, a més, una conseqüència tècnica de la implementació escollida que convé fer explícita: la funció `main` es proporciona dins d'una capçalera que el codi del joc inclou, de manera que és la unitat de compilació del joc la que efectivament la compila. És una solució que funciona i que manté el punt d'entrada sota control del motor, però que implica que la capçalera només es pot incloure des d'un únic fitxer del projecte del joc.

4.4 Gestió explícita de memòria

Aquesta és la decisió més característica del motor i la que millor il·lustra la tesi que travessa el treball: que les fonts consultades, tot i respondre a preocupacions diferents, convergeixen en la mateixa solució.

4.4.1 Problema i alternatives

Cada subsistema del motor manté un estat intern que ha de viure mentre duri l'execució. Cal decidir qui el reserva, on resideix i quan es allibera.

L'opció immediata és que cada subsistema reservi el seu estat pel seu compte mitjançant l'assignador general, i que l'alliberi en tancar-se. És senzilla i no requereix cap coordinació, però reparteix l'estat del motor per tot el munt de memòria, obliga cada subsistema a dependre d'un assignador concret i no dona al motor cap control sobre la ubicació ni sobre el moment de l'alliberament.

4.4.2 La convergència de dues exigències

La solució adoptada respon a dues exigències de procedència independent.

La primera prové de l'arquitectura de motors. Com s'ha exposat al capítol 2, les dependències entre subsistemes imposen un ordre d'arrencada, i la recomanació és fer aquest ordre explícit mitjançant funcions d'arrencada i tancament que s'invoquen des d'un únic lloc, en lloc de confiar-lo als mecanismes implícits del llenguatge [1].

La segona prové de l'API gràfica. Vulkan no reserva memòria en nom de l'aplicació, i això li imposa una convenció de crida característica: per obtenir una llista d'elements, la primera crida es fa amb el punter de destinació nul i retorna el nombre d'elements disponibles; la segona, amb l'espai ja reservat per l'aplicació, l'omple [5]. Qui reserva l'emmagatzematge ha de saber-ne primer la mida.

El patró d'inicialització del motor és la composició d'ambdues. Cada funció d'inicialització s'invoca dues vegades: la primera amb el punter d'estat nul, cas en què es limita a informar de la mida que necessita; entre les dues crides, l'aplicació reserva aquesta quantitat de memòria; i la segona crida ja rep el bloc i hi construeix l'estat. Aquesta seqüència es repeteix per a cada subsistema, en l'ordre que les dependències imposen, dins d'una única funció.

La memòria surt d'un assignador lineal, que reserva un bloc de 64 MiB en iniciar-se i el reparteix de manera incremental sense alliberar-ne mai peces individuals. L'estat de tots els subsistemes acaba, doncs, en una única regió contigua amb un cicle de vida igual al del procés.

4.4.3 Criteri i decisió

El resultat és que ni la ubicació de l'estat ni el seu moment d'alliberament són decisions del subsistema, sinó de qui el condueix. Un subsistema que no reserva la seva pròpia memòria no pot imposar cap assignador a la resta del motor, no pot fragmentar el munt i no pot sobreviure

al bloc que se li ha lliurat. S'obté, a més, una propietat associada a l'ús d'assignadors d'aquesta mena: la proximitat en memòria de l'estat que s'utilitza conjuntament [1].

4.4.4 Cost de la decisió

Aquest patró té tres costos que convé enunciar.

El primer és que cada subsistema desa el bloc rebut en un punter estàtic de la seva unitat de compilació, cosa que fa ergonòmic el patró —cap funció no ha de rebre un paràmetre de context— però limita el motor a una única instància de cada subsistema per procés. No hi pot haver dos renderitzadors ni una cua d'esdeveniments per fil d'execució.

El segon és que la reserva no imposa cap alineació: retorna el següent byte lliure. Els estats dels subsistemes queden correctament alineats per la mida de les estructures que els precedeixen, no per garantia de l'assignador. És una limitació coneguda i es discuteix al capítol 7.

El tercer és que la xifra de 64 MiB descriu l'estat dels subsistemes, però no la petjada real del motor: els subsistemes que reserven memòria durant la seva inicialització —el registre obre un fitxer, el sistema d'esdeveniments crea vectors dinàmics— ho fan fora d'aquest bloc.

4.5 Separació entre la part independent de l'API i la implementació

4.5.1 Problema i alternatives

Perquè la decisió documentada a la secció sobre l'API gràfica no quedés en una declaració d'intencions, calia que el motor no assumís Vulkan arreu. Es tracta de decidir on se situa la frontera entre allò que qualsevol renderitzador ha de fer i allò que només té sentit sobre una API concreta.

4.5.2 Criteri i decisió

La frontera es materialitza en una estructura de punters a funció que la part independent de l'API invoca i que una funció de selecció omple segons la implementació activa. Aquesta estructura declara nou operacions: inicialització, tancament, notificació de canvi de mida, inici i final de *frame*, actualització de l'estat global, actualització d'un objecte, i creació i destrucció de textures.

El criteri que governa aquesta llista és el nivell d'abstracció. Les operacions parlen de *frames*, d'objectes i de textures, no de primitives de la GPU: no hi ha cap entrada per crear una canonada gràfica, iniciar una passada de renderitzat o vincular un conjunt de descriptors. Aquesta elecció és deliberada. Una interfície de gra fi que exposés els conceptes de Vulkan hauria estat més flexible, però hauria traslladat el model d'aquesta API a la part que se suposava independent, i qualsevol implementació alternativa hauria hagut d'emular-lo.

La conseqüència és que la part independent pot mantenir les matrius de projecció i de vista i un punter a la textura activa sense saber què és un conjunt de descriptors, i que tot allò específic de l'API —memòria de dispositiu, *command buffers*, sincronització, disposicions d'imatge— queda confinat rere aquestes nou funcions.

4.5.3 Cost de la decisió

Els punters a funció no reben cap paràmetre de context. La implementació manté, per tant, el seu propi estat de manera estàtica, cosa que simplifica les signatures a canvi d'impedir que hi hagi dues implementacions actives simultàniament. Per a un motor que selecciona la implementació en temps de construcció, la limitació no té conseqüències pràctiques.

Hi ha, a més, una fuita coneguda i documentada al codi. La funció que estableix la matriu de vista s'exporta fora del motor perquè l'aplicació de proves hi pugui empenyer la matriu de

la càmera, i la capçalera la marca explícitament com una solució provisional. La via correcta seria que el joc escrivís un estat de més alt nivell i que el renderitzador el llegís del paquet de renderitzat que ja rep a cada *frame*, però aquest paquet encara no transporta informació d'escena. És una excepció reconeguda al criteri que la resta de la secció defensa.

4.6 Model de recursos: referències, identificadors i generacions

4.6.1 Problema i alternatives

Els recursos que el motor carrega des de disc —textures i, posteriorment, materials— es comparteixen entre diverses parts de l'escena i tenen un cost d'ocupació a la GPU. Cal decidir quan se'n reclama la memòria.

Aquesta qüestió es planteja habitualment com la gestió del cicle de vida dels recursos, i cada recurs hi té requisits propis: alguns han de romandre en memòria durant tota la partida, altres tenen un cicle de vida lligat al d'un nivell concret, i altres encara més breu [1]. La solució establerta és el recompte de referències, amb la descàrrega del recurs quan el recompte arriba a zero [1].

4.6.2 Criteri i decisió

El motor implementa aquest esquema mitjançant una taula de dispersió que associa el nom del recurs amb una estructura que en registra el recompte de referències, un identificador i un indicador d'alliberament automàtic. Adquirir un recurs n'incrementa el recompte i, si encara no existeix, en desencadena la càrrega; alliberar-lo el decrementa.

L'indicador d'alliberament automàtic és el que aporta la política de cicle de vida. Un recurs adquirit amb aquest indicador desactivat es comporta com un actiu global [1]: roman carregat encara que el recompte arribi a zero. Amb l'indicador actiu, es descarrega en deixar de tenir referències. És una versió d'un sol bit d'una taxonomia de cicles de vida més detallada [1], però n'expressa la distinció essencial.

A aquest esquema s'hi afegeix un segon mecanisme. Cada recurs manté, a més de l'identificador, un camp de generació que s'incrementa cada vegada que el contingut es torna a carregar. Un punter per si sol no permet saber que allò que apunta ha estat substituït; la parella formada per l'identificador i la generació sí que ho permet.

Aquest mecanisme té una utilitat concreta al renderitzador. Els conjunts de descriptors que vinculen una textura a un *shader* han de reescriure's quan la textura canvia, però reescriure'ls a cada *frame* seria innecessari. La implementació desa, per a cada descriptor, l'identificador i la generació de la textura que hi té vinculada, i només emet l'operació d'actualització quan algun dels dos difereix del valor actual del recurs.

4.6.3 Cost de la decisió

El recompte de referències és manual: si una part del motor adquireix un recurs i no l'allibera, aquest no es descarregarà mai. El llenguatge no ofereix cap mecanisme que ho previngui, cosa que és conseqüència directa de la decisió documentada a la secció sobre el llenguatge.

Hi ha, a més, una limitació estructural. Aquest esquema s'implementa de manera independent per a cada tipus de recurs, de manera que el sistema de textures i el de materials repeteixen la mateixa estructura —taula de dispersió, vector de registres, recompte i indicador d'alliberament— sense compartir-ne cap. Falta la capa genèrica de gestió de recursos que unificaria l'accés a qualsevol tipus d'actiu [1]. Es tracta d'una de les línies de continuació que es recullen al capítol 7.

[Revisar aquesta secció quan el sistema de materials estigui integrat. La descripció del recompte de referències i de les generacions és estable, però la part relativa als materials descriu un sistema encara no incorporat al camí d'execució.]

5 Disseny i implementació

5.1 Arquitectura general i cicle de vida

[Diagrama de capes + seqüència d'arrencada + cicle d'un frame. Aquesta secció és el mapa; les quatre següents són el detall.]

5.2 Gestió de memòria

[hmemory amb etiquetes, assignador lineal, i l'assignador de frame implementat des de [GEA §6.2.1.3]. Marcar clarament què és aportació pròpia.]

5.3 Sincronització CPU/GPU i el bucle de *frame*

[La secció amb més valor de defensa que teniu. Tres sincronitzacions diferents (CPU per davant de GPU, dibuix després d'adquisició, presentació després de dibuix) i per què els vectors de semàfors tenen mides diferents. Explicar el defecte trobat: quan `image_count` i `max_frames_in_flight` divergeixen — cosa que no passa al maquinari del tutorial — la indexació incorrecta del semàfor de senyalització provoca l'error. Trobar-lo va exigir entendre el model, no transcriure'l.]

5.4 Sistemes de recursos

[Textures, materials, geometria. El format `.hmt` com a primer actiu dirigit per dades. Traçar el camí sencer: fitxer → configuració → recurs → descriptor de GPU.]

5.5 Resta de subsistemes

[Taula resum: subsistema, responsabilitat, fitxers, estat. Remetre a l'Annex C per a l'API pública i al repositori per al codi.]

6 Resultats i discussió

6.1 Metodologia de mesura

[Com es mesura: instrumentació pròpia, nombre de repeticions, què es descarta (frames d'escalfament), maquinari i controlador exactes. Sense això, els resultats no són defensables.]

6.2 Assoliment dels objectius funcionals

[Per a cada objectiu específic de §1.2, l'evidència que s'ha assolit. Captures del motor en funcionament. Aquesta secció és la que tanca el cercle amb «los objetivos están bien delimitados».]

6.3 Experiments

[Dos o tres n'hi ha prou. Candidats, per ordre de valor:

- 1. **Staging buffer enfront d'escriptura directa sobre memòria unificada.** Sobre GPU integrada, `DEVICE_LOCAL` i `HOST_VISIBLE` coincideixen, de manera que la còpia intermèdia pot ser sobrecost pur. La mateixa especificació preveu el cas d'un únic munt [VkSpec §3.2].*
- 2. **max_frames_in_flight a 1, 2 i 3.** Temps de frame enfront de latència d'entrada.*
- 3. **Modes de presentació MAILBOX i FIFO.***
- 4. **Reescriptura de descriptors cada frame enfront de reescriptura només quan canvia generation.***

Els experiments s'executen sobre els dos equips de la Taula 5, de manera que cada resultat es pot llegir en contrast entre una arquitectura de memòria unificada i una de separada.

Cada un: hipòtesi, muntatge, gràfic, lectura. Un gràfic amb una lectura honesta val més que quatre gràfics sense.]

6.4 Discussió

[Què signifiquen els resultats i què impliquen per al disseny del motor. Interessa especialment on el consell general de la literatura no s'ajusta al cas concret mesurat — per exemple, si la còpia intermèdia resulta innecessària sobre memòria unificada.]

6.5 Limitacions de l'avaluació

[Una sola màquina, un sol controlador, escena mínima. Reconèixer-ho enforteix la secció; amagar-ho la debilita si algú del tribunal ho pregunta.]

7 Conclusions

7.1 Conclusions del treball realitzat

[Tancar contra els objectius de §1.2, un per un.]

7.2 Punts forts i punts febles

[Exigit per la plantilla. Els punts febles escrits per un mateix són el senyal més fort de comprensió que es pot donar — i desactiven la meitat de les preguntes hostils possibles.]

7.3 Limitacions

[Un sol fil d'execució, un sol render pass, sense alineació a l'assignador, geometria limitada. Les seccions «Known limitations» de docs/ ja contenen aquest material.]

7.4 Línies de continuació

[Sistema de tasques concurrents, cua de renderitzat, subassignador de memòria de GPU, sistema de recursos genèric. Cada una amb la font que en descriu el disseny.]

Referències

[Estil numerat [1], [2]... referenciat de manera creuada des del text. El fitxer refs.bib conté una base inicial. Verificar cada entrada: any, editorial, ISBN, URL i data de consulta. Val 0,5 punts, i és el criteri més barat de tot el rúblic.]

- [1] J. Gregory, *Game Engine Architecture*, 3r ed. CRC Press, 2018.
- [2] C. Everitt, «OpenGL Efficiency: AZDO». Consulta: 8 de setembre de 2026. [En línia]. Disponible a: <https://www.khronos.org/assets/uploads/developers/library/2014-gdc/Khronos-OpenGL-Efficiency-GDC-Mar14.pdf>
- [3] T. Olson, «Vulkan 101». 2016.
- [4] G. Riguer, «Mantle Programming Guide and API Reference». març de 2015.
- [5] Khronos Vulkan Working Group, «Vulkan 1.4.361 — A Specification (with all registered extensions)». 2026. Consulta: 2 de setembre de 2026. [En línia]. Disponible a: <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html>
- [6] Unity Technologies, «Unity Documentation».
- [7] Epic Games, «Unreal Engine Documentation».
- [8] Godot Engine contributors, «Godot Engine Documentation».
- [9] T. Vroman, «Kohi Game Engine / Shadows of Illumina (Vulkan Game Engine and Game Written in C)». Consulta: 9 de setembre de 2026. [En línia]. Disponible a: <https://www.youtube.com/playlist?list=PLv8Ddw9K0JPg1BEO-RS-0MYs423cvLVtj>
- [10] T. Vroman, «kohi — A game engine made as part of the Kohi Game Engine series, written in C and Vulkan». Consulta: 9 de setembre de 2026. [En línia]. Disponible a: <https://github.com/travisvroman/kohi>
- [11] C. Muratori, «Handmade Hero».
- [12] B. Karadžić, «bgfx — Cross-platform rendering library».
- [13] S. Willems, «Vulkan C++ examples and demos».
- [14] The Khronos Group, «MoltenVK — A Vulkan Portability implementation over Apple's Metal graphics framework». Consulta: 10 de setembre de 2026. [En línia]. Disponible a: <https://github.com/KhronosGroup/MoltenVK>
- [15] B. Stroustrup, *The C++ Programming Language*, 4t ed. Addison-Wesley, 2013.
- [16] R. Fabian, *Data-Oriented Design*. 2018.

Annexos

[

Annex A *Mapatge de decisions de disseny a fonts primàries. (Traducció i adaptació de docs/analysis/concepts-and-sources.typ.)*

Annex B *Manual de compilació i execució.*

Annex C *Referència de l'API pública dels sistemes.*

Annex D *Resultats complets de les mesures. (Al cos, només els gràfics.)*

Annex E *Planificació detallada i diagrama de Gantt.*

]